

Foundations of Machine Learning (AI2000/AI5000): Assignment-1
Released on: 20th August, 2025
Deadline: 30th August, 2025, EoD (11:59PM, IST)
Maximum Marks: 30M

A. Instructions

- Answer all questions. We encourage best coding practices by not penalizing (i.e. you may not get full marks if you make it difficult for us to understand). Hence, use intuitive names for the variables, and comment your code wherever necessary. Use the markdown cells in the notebook for briefly explaining the objective of a code cell or reporting the observations if asked in the question.
- It is expected that you work on these problems individually. If you have any doubts please contact the TA or the instructor no later than 2 days prior to the deadline.
- Use of LLMs are strictly prohibited. If found, you will be heavily penalized in the respective questions.
- You may use built-in implementations only for the basic functions such as `sqrt`, `log`, etc. from libraries such as `numpy`. For plots, you may use `matplotlib` and generate clear plots that are complete and easy to understand. For sampling images, in Q1 you may use `numpy multi-variate gaussian` function. Apart from these any other high-level functionalities are required to be implemented by the students.
- **Students are required to submit a Python notebook titled `A1.<your-roll-number>.ipynb`, together with a PDF document containing all relevant mathematical derivations as specified in the assignment.**

B. Problems

1. Parameter Estimation (15M)

Consider the MNIST dataset restricted to the following digit space:

$$\mathcal{K} = \{2, 4, 7\}.$$

For each digit $k \in \mathcal{K}$, you are provided with a dataset

$$\mathcal{D}_k = \{x_1^{(k)}, \dots, x_{N_k}^{(k)}\}, \quad x_i^{(k)} \in \mathbb{R}^{784},$$

where each $x_i^{(k)}$ represents the vectorized 28×28 grayscale image (784 pixel intensities).

Assume that, for each digit k , the dataset is generated by sampling i.i.d. from a 784-dimensional multivariate Gaussian distribution

$$x \mid (y = k) \sim \mathcal{N}(\overline{\mu}_k, \Sigma_k), \quad \mu_k \in \mathbb{R}^{784}, \quad \Sigma_k \in \mathbb{R}^{784 \times 784}$$

Note: Before computing the estimates, please make sure to normalize the pixel intensity values to the range $[0, 1]$ by dividing with 255.0

(a) Maximum Likelihood Estimation (MLE) (4M)

1. Derive the Maximum Likelihood Estimates (MLEs) of the mean vector μ_k and covariance matrix Σ_k for each digit $k \in \mathcal{K}$.
2. Are the above estimates unbiased? If not, compute the unbiased estimates.

(b) Maximum A Posteriori Estimate (MAP) (7M)

Now you are free to explore and come up with your own prior for $\overline{\mu}_k$ parameter (*Think of some multi-variate distribution since μ_k is a vector*) and assume no prior necessarily required to impose on Σ_k . In this setting, derive the Maximum A Posteriori (MAP) estimate of the mean vector μ_k and covariance matrix Σ_k for each digit $k \in \mathcal{K}$.

(c) Sampling and Comparison (4M)

Sampling (MLE): For each desired synthetic image of class k , draw

$$x^{(\text{MLE sample})} \sim \mathcal{N}(\overline{\mu}_k^{\text{MLE}}, \Sigma_k^{\text{MLE}})$$

Clip to $[0, 1]$, reshape to 28×28 , and generate 10 images and display them in your submission.

Sampling (MAP): For each desired synthetic image of class k , draw

$$x^{(\text{MAP sample})} \sim \mathcal{N}(\overline{\mu}_k^{\text{MAP}}, \Sigma_k^{\text{MAP}})$$

Clip to $[0, 1]$, reshape to 28×28 , and generate 10 images and display them in your submission.

Comment (brief, per class): Which set (MLE or MAP) produces more visually acceptable digits given the sample size n_k ? Provide a suitable explanation (1–2 sentences) referring to prior influence.

Note: Include all the mathematical derivation steps for the MLE and MAP estimates in a separate document (as mentioned in the above instructions given in section A). This will also be considered for evaluation in this question.

2. Linear Regression (15M)

You are provided with a dataset containing **noisy observations** of a polynomial function. Your task is to implement and analyze **Polynomial Regression** using different approaches and study the impact of model complexity.

Part (a): Polynomial Regression using Gradient Descent (8M)

1. Implement a **polynomial regression model** with the degree of your choice to fit the data by training the model parameters (coefficients) using **gradient descent** optimization technique.

For the gradient descent method, you are responsible to choose appropriate hyperparameters (for better convergence) such as:

- Learning rate (η)
- Number of iterations (epochs)

After training, you are expected to provide the following plots

- Training Loss (Mean-Squared Error (MSE)) vs. iterations (to show convergence)
- Fitted curve overlaid on given data points you used for training

Report the **converged polynomial coefficients/weights** obtained after training.

2. Next try experimenting with at least 4-5 polynomial degrees and repeat step - 1.

Part (b): Comparison with Normal Equation (4M)

1. Compute the coefficients of all the polynomial degrees for linear regression model which you have experimented in part(a) using the **Normal Equation** discussed in class.
2. Compare the coefficients obtained from gradient descent (part(a)) with those obtained from the normal equation and check the following:
 - Whether both approaches converge approximately to the same solution programmatically.
 - Reasons for any differences observed (e.g., convergence criteria, learning rate, or numerical issues etc.).

Part (c): Understanding Overfitting and Underfitting (3M)

Based on the analysis from part (a) and part (b), discuss the range of polynomial degrees for which **underfitting** and **overfitting** occur. Clearly identify the **best-fit polynomial(s)** and justify your choice with appropriate reasoning, supported by quantitative analysis wherever necessary.

Additionally, include explanatory notes in markdown cells to demonstrate how your observations relate to underfitting, overfitting, or a well-balanced fit.